WebGL

dr inż. Robert Krupiński

html

```
<!DOCTYPE html>
<html>

<head>
    <title>WebGL</title>
    <meta charset="UTF-8">

    <script type="text/javascript" src="lib/webgl-debug.js"></script>

    <style>
        body {
            background-color: grey;
        }

        canvas {
            background-color: white;
        }
    </style>
</head>

<body>

    <canvas id="id_Canvas" width="400" height="300">
        Your browser does not support the HTML5 canvas element.
    </canvas>
```


https://registry.khronos.org/webgl/sdk/devtools/src/debug/webgl-debug.js

html

```
27      <script type="module">
28
29          import MyWebGL from "./MyWebGL.js"
30
31          // Vertex shader program
32          const astrVertexShader =
33              "attribute vec4 a_Position;\n" + // attribute variable
34              "attribute float a_PointSize; \n" +
35              "void main() {\n" +
36              "  gl_Position = a_Position;\n" + // Set the vertex coordinates of the point
37              "  gl_PointSize = a_PointSize;\n" + // Set the point size
38              "}\n";
39
40          // Fragment shader program
41          const astrFragmentShader =
42              "precision mediump float;\n" +
43              "uniform vec4 u_FragColor;\n" +
44              "void main() {\n" +
45              "  gl_FragColor = u_FragColor;\n" + // Set the point color
46              "}\n";
```

html

```
48      window.onload = () => {
49
50          const aMyWebGL = new MyWebGL();
51          // Retrieve <canvas> element
52          const aCanvas = document.getElementById("id_Canvas");
53          // Get the rendering context for WebGL
54          const gl = aMyWebGL.initGL(aCanvas, true)
55          if (!gl) {
56              console.error("Failed to get the rendering context for WebGL");
57              return
58          }
59
60          const aProgram = aMyWebGL.initShadersAndMakeCurrent(astrVertexShader, astrFragmentShader);
61          if (!aProgram) {
62              console.error("!aProgram");
63              return
64          }
65
66          const a_Position = gl.getAttribLocation(aProgram, "a_Position");
67          if (0 > a_Position) {
68              console.error("Failed to get the storage location of a_Position")
69              return
70          }
71          const a_PointSize = gl.getAttribLocation(aProgram, "a_PointSize");
72          if (0 > a_PointSize) {
73              console.error("Failed to get the storage location of a_PointSize")
74              return
75          }
```

html

```
76              // Get the storage location of u_FragColor
77              const u_FragColor = gl.getUniformLocation(aProgram, "u_FragColor");
78              if (!u_FragColor) {
79                  console.error("Failed to get the storage location of u_FragColor")
80                  return
81              }
82
83              // Pass vertex position to attribute variable
84              gl.vertexAttrib3f(a_Position, 0.5, 0.0, 0.0)
85              gl.vertexAttrib1f(a_PointSize, 15.0)
86              // Pass the color of a point to u_FragColor variable
87              gl.uniform4f(u_FragColor, 0.0, 1.0, 0.0, 1.0)
88              // Set clear color
89              gl.clearColor(0.0, 0.0, 0.0, 1.0)
90              gl.clear(gl.COLOR_BUFFER_BIT)
91              gl.drawArrays(gl.POINTS, 0, 1)
92          }
93
94      </script>
95
96  </body>
97
98  </html>
```

MyWebGL.js

```
2   class MyWebGL {
3
4       gl;
5       VShader;
6       FShader;
7       Program;
8
9       static createGL(aCanvas, avAttribs) {
10          if (!aCanvas) {
11              console.error("createGL:!aCanvas", aCanvas)
12              return null
13          }
14          const avstrNames = ["webgl", "experimental-webgl", "webkit-3d", "moz-webgl"];
15          let gl = null, n = avstrNames.length;
16
17          for (let i = 0; i < n; ++i) {
18              try {
19                  gl = aCanvas.getContext(avstrNames[i], avAttribs)
20              } catch (e) {
21              }
22              if (gl) {
23                  break
24              }
25          }
26          return gl
27      }
```

MyWebGL.js

```
29      initGL(aCanvas, abDebug) {
30          let gl = MyWebGL.createGL(aCanvas);
31          if (!gl) {
32              return null
33          }
34          if (abDebug) {
35              gl = WebGLDebugUtils.makeDebugContext(gl)
36          }
37          this.gl = gl
38          return gl
39      }
40
41      initShadersAndMakeCurrent(astrVertexShader, astrFragmentShader) {
42          const gl = this.gl;
43          if (!gl) {
44              console.error("initShadersAndMakeCurrent:!gl")
45              return null
46          }
47          const aProgram = this.initShaders(astrVertexShader, astrFragmentShader);
48          if (!aProgram) {
49              console.error("initShadersAndMakeCurrent:this.initShaders failed")
50              return null
51          }
52          gl.useProgram(aProgram)
53          return aProgram
54      }
```

MyWebGL.js

```
56      initShaders(astrVertexShader, astrFragmentShader) {
57          const gl = this.gl;
58          if (!gl) {
59              console.error("initShaders:!gl")
60              return null
61          }
62          if (0 >= astrVertexShader.length) {
63              console.error("initShaders:astrVertexShader empty")
64              return null
65          }
66          if (0 >= astrFragmentShader.length) {
67              console.error("initShaders:astrFragmentShader empty")
68              return null
69          }
70          const aVShader = this.createShaderObject(gl.VERTEX_SHADER, astrVertexShader);
71          if (!aVShader) {
72              console.error("initShaders:aVShader undefined")
73              return null
74          }
75          const aFShader = this.createShaderObject(gl.FRAGMENT_SHADER, astrFragmentShader);
76          if (!aFShader) {
77              console.error("initShaders:aFShader undefined")
78              gl.deleteShader(aVShader)
79              return null
80          }
81          const aProgram = this.createProgramObject(aVShader, aFShader);
82          if (!aProgram) {
83              console.error("initShaders:createProgram failed")
84              gl.deleteShader(aVShader)
85              gl.deleteShader(aFShader)
86              return null
87          }
88          this.VShader = aVShader
89          this.FShader = aFShader
90          this.Program = aProgram
91          return aProgram
92      }
```

MyWebGL.js

```
94      createShaderObject(aeShaderType, astrSource) {
95          const gl = this.gl;
96          if (!gl) {
97              console.error("createShaderObject:!gl")
98              return null
99          }
100         const aShader = gl.createShader(aeShaderType);    // Create shader object
101         if (!aShader) {
102             console.error("createShaderObject:createShader failed")
103             return null
104         }
105         gl.shaderSource(aShader, astrSource)    // Set the shader program
106         gl.compileShader(aShader)               // Compile the shader
107         if (!gl.getShaderParameter(aShader, gl.COMPILE_STATUS)) {  // Check the result of compilation
108             console.error(`Failed to compile shader: ${gl.getShaderInfoLog(aShader)}`)
109             gl.deleteShader(aShader)
110             return null
111         }
112         return aShader
113     }
```

MyWebGL.js

```
115     createProgramObject(aVShader, aFShader) {
116         const gl = this.gl;
117         if (!gl) {
118             console.error("createProgramObject:!gl")
119             return null
120         }
121         if (!aVShader) {
122             console.error("createProgramObject:aVShader undefined")
123             return null
124         }
125         if (!aFShader) {
126             console.error("createProgramObject:aFShader undefined")
127             return null
128         }
129         const aProgram = gl.createProgram();   // Create a program object
130         if (!aProgram) {
131             console.error("createProgramObject:createProgram failed")
132             return null
133         }
134         gl.attachShader(aProgram, aVShader)        // Attach the shader objects
135         gl.attachShader(aProgram, aFShader)
136         gl.linkProgram(aProgram)                   // Link the program object
137         if (!gl.getProgramParameter(aProgram, gl.LINK_STATUS)) { // Check the result of linking
138             console.error(`Failed to link program: ${gl.getProgramInfoLog(aProgram)}`)
139             gl.deleteProgram(aProgram)
140             return null
141 
142         }
143         return aProgram
144     }
145 }
146 
147 export default MyWebGL;
```

File location

- www.rmaes.com/students/2023_2024/WebGL.pdf